

Day 10 复习总表：异步编程基础（上）——回调函数与 Promise

Step 项	完成事项	对应代码 / 指令	代码详解
Step 2 同步 vs 异步	用 <code>setTimeout</code> 直观感受异步不阻塞	<pre>js console.log("第一步"); setTimeout(() => console.log("第二步"), 2000); console.log("第三步");</pre>	<code>setTimeout</code> 是异步任务，注册后立即"放行"，不等待 - 实际输出顺序：第一步 → 第三步 → 第二步，和书写顺序不同
Step 3 回调函数	用回调处理"数据准备好之后"的逻辑	<pre>js function fetchUserData(callback) { setTimeout(() => callback(userData), 1500); } fetchUserData(data => console.log(data));</pre>	- 函数可以作为参数传递给另一个函数（回调） - 数据准备好后，由内部逻辑主动调用回调，把结果传出去 "什么时候执行回调"由被调用的函数自己决定，不是同步立即执行
Step 4 回调地狱问题	通过 3 层嵌套示例直观感受维护难度	<pre>js step1(() => { step2(() => { step3(() => {...}); }); });</pre>	- 每多一层异步依赖，代码就多缩进一层，形成"厄运金字塔" - 问题：可读性差、错误处理分散、难以插入/调整步骤
Step 5 Promise 基础语法	用 Promise 重写同样的异步逻辑	<pre>js new Promise((resolve, reject) => { if (success) resolve(data); else reject(err); }); promise.then(data => {...}).catch(err => {...});</pre>	<code>resolve</code>/<code>reject</code> 分别对应成功/失败两条路径 <code>.then()</code> 处理成功结果，<code>.catch()</code> 处理失败结果 - 链式写法始终保持同一层缩进，比嵌套回调更易读
Step 6 Git 日常推送	完成本日代码的提交和推送	<pre>bash git add script.js git commit -m "..."</pre> <pre>git push</pre>	- 与 Day 9 首次推送不同，这里不用再写 <code>-u origin main</code> - 追踪关系已建立，Git 自动知道推到哪
Step 7 Code Review + 自评	5 道概念题检验理解	考察问题： 1. 异步为何不阻塞 2. 回调函数定义 3. 回调地狱本质 4. resolve/reject 调用时机 5. then/catch 对应关系	- 异步任务注册后立即放行，不等待 - 回调 = 把函数当参数传递，由对方决定何时调用 - 回调地狱 = 嵌套过深导致难读、难维护、错误处理棘手 - resolve=成功时调用，reject=失败时调用 - then↔resolve, catch↔reject

本日核心概念速查

概念	一句话记忆
异步不阻塞	注册异步任务后立即往下走，不会"傻等"结果
回调函数	把函数交给别人，让对方在合适的时机替你调用
回调地狱	嵌套越深越难读，是回调函数的最大痛点
Promise 的两条路	<code>resolve</code> 走向 <code>.then()</code> ， <code>reject</code> 走向 <code>.catch()</code>
Promise 的优势	链式写法替代嵌套，同一层缩进读到底